

Core Java

Interview Questions & Answers

Prepared By : Krishna Agrawal

Link : <https://www.linkedin.com/in/staylearner-krishna-agrawal/>

Topics Covered

- THREADING
- Collections
- EXCEPTION HANDLING
- GARBAGE COLLECTOR
- OOPS Concepts
- SERIALIZATION
- Immutable Class and String
- Basic Core Java

THREADING

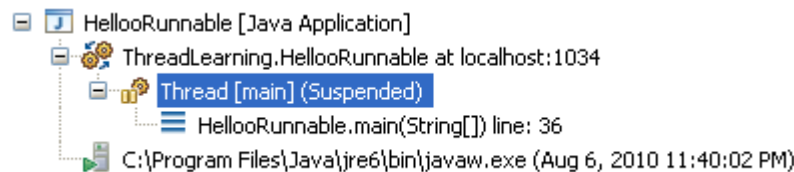
What is a Thread?

In Java, "thread" means two different things:

- ❖ An instance of class `java.lang.Thread`.
- ❖ A thread of execution.

An instance of `Thread` is just an object. Like any other object in Java, it has variables and methods, and lives and dies on the heap. But a thread of execution is an individual process (a "lightweight" process) that has its own call stack. In Java, there is one thread per call stack—or, to think of it in reverse, one call stack per thread. Even if we don't create any new threads in our program, threads are running at back.

The `main()` method, that starts the whole ball rolling, runs in one thread, called the main thread. If we looked at the main call stack, we would see that `main()` is the first method on the stack—the method at the bottom. But as soon as you create a new thread, a new stack materializes and methods called from that thread run in a call stack that's separate from the `main()` call stack.



What are the advantages or usage of threads?

Threads often result in simpler programs.

- In sequential programming, updating multiple displays normally requires a big while-loop that performs small parts of each display update. Unfortunately, this loop basically simulates an operating system scheduler. In Java, each view can be assigned a thread to provide continuous updates.
- Programs that need to respond to user-initiated events can set up service routines to handle the events without having to insert code in the main routine to look for these events.

Threads provide a high degree of control.

- Imagine launching a complex computation that occasionally takes longer than is satisfactory. A "watchdog" thread can be activated that will "kill" the computation if it becomes costly, perhaps in favor of an alternate, approximate solution. Note that sequential programs must muddy the computation with termination code, whereas, a Java program can use thread control to non-intrusively supervise any operation.

Threaded applications exploit parallelism.

- A computer with multiple CPUs can literally execute multiple threads on different functional units without having to simulating multi-tasking ("time sharing").
- On some computers, one CPU handles the display while another handles computations or database accesses, thus, providing extremely fast user interface response times.

What is difference between thread and process?

Differences between threads and processes are:-

1. Threads share the address space of the process that created it; processes have their own address.
2. Threads have direct access to the data segment of its process; processes have their own copy of the data segment of the parent process.
3. Threads can directly communicate with other threads of its process; processes must use inter-process communication to communicate with sibling processes.
4. Threads have almost no overhead; processes have considerable overhead.
5. New threads are easily created; new processes require duplication of the parent process.
6. Threads can exercise considerable control over threads of the same process; processes can only exercise control over child processes.
7. Changes to the main thread (cancellation, priority change, etc.) may affect the behavior of the other threads of the process; changes to the parent process do not affect child processes.

What is the difference between preemptive scheduling and time slicing?

- ❖ Under preemptive scheduling, the highest priority task executes until it enters the waiting or dead states or a higher priority task comes into existence.
- ❖ Under time slicing, a task executes for a predefined slice of time and then reenters the pool of ready tasks. The scheduler then determines which task should execute next, based on priority and other factors.

Does each thread have its own thread stack?

Yes each thread has its own call stack. For e.g.

```
Thread t1 = new Thread();
Thread t2 = new Thread();
Thread t3 = t1
```

In the above example t1 and t3 will have the same stack and t2 will have its own independent stack.